

HII CONFIGURATION PROCESSING AND BROWSER PROTOCOL

35.1 Introduction

This section describes the data and APIs used to manage the system's configuration: the actual data that describes the knobs and settings.

35.1.1 Common Configuration Data Format

The configuration data is stored as name / value string pairs. As in e.g. HTML, the name and value are separated by '=' and the pairs are separated one from the next by '&'. The configuration data structures are thus variable length UNICODE (UCS-2) strings.

Certain names and values have limitations on their syntax to manage routing and to enable extended support for common storage mechanisms.

35.1.2 Data Flow

There is a two-way flow through the hierarchy of drivers and protocols that parallels the flow in other parts of HII. Initially, the flow is from the drivers up to the HII database and on to configuration applications. When changes to configuration are accepted, the flow reverses itself, going from the configuration applications through the HII database protocols back to the drivers through separate protocols.

The flow from driver up consists of the current and alternative (default) configurations. The flow down from the configuration applications consists of changed configurations.

The protocol managed by the HII Database is known as the EFI HII Configuration Routing Protocol, while the one presented by the drivers themselves is known as the EFI HII Configuration Access Protocol. The HII Configuration Routing Protocol is the only one that outside callers should invoke.

35.2 Configuration Strings

The configuration strings follow the same general format as HTTP argument strings, which is to say '&' separated name / value pairs. The name and value are separated by '='. The strings are a subset of full HTML argument strings and do not require quoting, the '%' character sequences used to insert spaces, ampersands, equal signs, and the like into HTTP argument strings.

35.2.1 String Syntax

Assumptions are typical for BNF with the following extensions

Characters in single quotes, e.g. 'a', indicate terminals.

Square brackets immediately followed by a number *n* indicate that the contents are to be repeated *n* times, so ['a']4 would be "aaaa".

An italicized non-terminal, e. g. <All Printable ASCII Characters> is used to indicate a set of terminals whose definition is outside the scope of this document.

The syntax for configuration strings is as follows.

35.2.1.1 Basic forms

```
<Dec19> ::= '1' | '2' | ... | '9'
<DecCh> ::= '0' | <Dec19>
<HexAf> ::= 'a' | 'b' | 'c' | 'd' | 'e' | 'f'
<Hex1f> ::= <Dec19> | <HexAf>
<HexCh> ::= <DecCh> | <HexAf>
<Number> ::= <HexCh>+
<Alpha> ::= 'a' | ... | 'z' | 'A' | ... | 'Z'
```

35.2.1.2 Types

```
<Guid> ::= <HexCh>32
<LabelStart> ::= <Alpha> | "_"
<LabelBody> ::= <LabelStart> | <DecCh>
<Label> ::= <LabelStart> [<LabelBody>]*
<Char> ::= <HexCh>4
<String> ::= [<Char>]+
<AltCfgId> ::= <HexCh>4
```

35.2.1.3 Routing elements

```
<GuidHdr> ::= 'GUID='<Guid>
<NameHdr> ::= 'NAME='<String>
<PathHdr> ::= 'PATH='<UEFI binary Device Path represented as
hex number>
<DescHdr> ::= 'ALTCFG='<AltCfgId>
<ConfigHdr> ::= <GuidHdr>'&'<NameHdr>'&'<PathHdr>
<AltConfigHdr> ::= <ConfigHdr> '&'<DescHdr>
```

35.2.1.4 Body elements

```

<ConfigBody> ::= <ConfigElement>*
<ConfigElement> ::= '&'<BlockConfig> | '&'<NvConfig>
<BlockName> ::= 'OFFSET='<Number>'&WIDTH='<Number>
<BlockConfig> ::= <BlockName>'&VALUE='<Number>
<RequestElement> ::= '&'<BlockName> | '&'<Label>
<NvConfig> ::= <Label>'='<String> | <Label>'='<Number>
    
```

35.2.1.5 Configuration strings

```

<ConfigRequest> ::= <ConfigHdr><RequestElement>*
<MultiConfigRequest> ::= <ConfigRequest>['&' <ConfigRequest>]*
<ConfigResp> ::= <ConfigHdr><ConfigBody>
<AltResp> ::= <AltConfigHdr><ConfigBody>
<ConfigAltResp> ::= <ConfigResp> ['&' <AltResp>]*
<MultiConfigAltResp> ::= <ConfigAltResp> ['&' <ConfigAltResp>]*
<MultiConfigResp> ::= <ConfigResp> ['&'<ConfigResp>]*
    
```

Notes:

The *<Number>* represents a data buffer and is encoded as a sequence of bytes in the format %02x in the same order as the buffer bytes reside in memory.

The *<Guid>* represents a hex encoding of GUID and is encoded as a sequence of bytes in the format %02x in the same order as the GUID bytes reside in memory.

The syntax for a *<Label>* is the C label (e.g. Variable) syntax.

The *<ConfigHdr>* provides routing information. The name field is required even if non-block storage is targeted. In these cases, it may be used as a way to distinguish like storages from one another when a driver is being used

The *<BlockName>* provides addressing information for managing block (e.g. UEFI Variable) storage. The first number provides the byte offset into the block while the second provides the length of bytes.

The *<PathHdr>* presents a hex encoding of a UEFI device path. This is not the printable path since the printable path is optional in UEFI and to enable simpler comparisons. The data is encoded as strings with the format %02x bytes in the same order as the device path resides in RAM memory.

The *<ConfigRequest>* provides a mechanism to request the current configuration for one or more elements.

The *<AltCfgId>* is the identifier of a configuration declared in the corresponding IFR.

The name 'GUID' is also used to separate *<String>* or *<ConfigRequest>* elements in the equivalent *Multi* version. That is:

```
*GUID=...&NAME=...&...&fred=12&GUID=...&NAME=...&...&goyle=11*
```

Indicates two *<String>*, with one ending with *fred=12*.

The following are reserved *<name>* s and cannot be used as names in a *<ConfigElement>* :

```

GUID
NAME
PATH
ALTCFG
OFFSET
    
```

(continues on next page)

(continued from previous page)

WIDTH
VALUE**35.2.1.6 Keyword strings**

```

<NamespaceId>      ::= 'NAMESPACE=' <String> '&'
<Keyword>          ::= 'KEYWORD=' <String> [ ':' <DecCh> (1/4) ]
<DataFilter>       ::= 'Buffer' | 'Numeric' [ ':' '1' | ':' '2' | ':' '4' | ':' '8' ]
                   | 'String' | 'Boolean' | 'Date' | 'Time'
<UsageFilter>      ::= 'ReadOnly' | 'ReadWrite'
<Filter>           ::= <UsageFilter> | <DataFilter> | <UsageFilter>
                   '&' <DataFilter>
<ValueRange>       ::= '&MAX=' <Number> '&MIN=' <Number> [ '&STEP=' <Number> ]
<ValueOption>      ::= '&OPTIONVALUE=' <Number> '&OPTIONSTRING=' <String>
                   [ '&VALUETYPE=' 'Numeric' [ ':' '1' | ':' '2' | ':' '4' | ':' '8' ] ]
<ValueAttribute>   ::= [ <ValueRange> ] [ <ValueOption> * ]
<Default>          ::= [ '&STANDARDDEFAULT=' <Number> ]
                   [ '&MFGDEFAULT=' <Number> ]
                   [ '&SAFEDEFAULT=' <Number> ]
<Display>          ::= '&DISPLAYNAME=' <String>
<DataType>         ::= '&DATATYPE=' <DataFilter>
<KeywordInfoFilter> ::= 'All' | [ 'DataType' ] [ 'ValueAttribute' ] [ 'Default' ]
                   [ 'DisplayName' ]
<Boolean>          ::= 'True' | 'False'
<KeywordInfoRequest> ::= 'KEYWORDINFO=' <KeywordInfoFilter>
<KeywordInfoResp>  ::= [ <DataType> ] [ <ValueAttribute> ] [ <Default> ] [ <Display> ]
<KeywordRequest>   ::= [ <PathHdr> '&' ] <Keyword>
                   [ '&' <KeywordInfoRequest> ] [ '&' <Filter> ]
<KeywordResp>      ::= <NamespaceId> <PathHdr> '&' <Keyword> '&VALUE=' <Number>
                   [ '&READONLY' ] [ <KeywordInfoResp> ]
<MultiKeywordRequest> ::= <KeywordRequest> [ '&' <KeywordRequest> ] *
<MultiKeywordResp>  ::= <KeywordResp> [ '&' <KeywordResp> ] *

```

NOTE: For Keyword definitions, see the UEFI Configuration Namespace Registry document on <http://uefi.org/uefi>.

HII Question Type	HII Keyword Data Type	Data information in <ValueAttribute>
Numeric	Numeric	ValueRanges
One-Of	Numeric	ValueOptions
Checkbox	Boolean	
String	String	ValueRanges
Ordered-List	Buffer	ValueOptions
Date	Date	
Time	Time	

The <NamespaceId> element is equivalent to the platform configuration language being used for the keyword definition*.

The <Keyword> element uses the 'KEYWORD=' name to designate that immediately following the reserved name is a string value associated with a configuration namespace keyword as defined in the Configuration NameSpace Registry document (<http://uefi.org/uefi>).

Typically, when a Keyword is defined, the value is a solitary string such as “BIOSVendor”. However, when certain Keywords are intended to represent a setting that may have multiple instances (e.g. ChipsetSATAPortEnable), that is when a “:<DecCh>(1/4)” suffix will be appended to the keyword definition. In that case, we might see something like: “ChipsetSATAPortEnable:5” if a particular platform had at least five SATA ports and one of the questions was represented by the aforementioned string. It would also be reasonable to expect that there might also be a “ChipsetSATAPortEnable:1” and a “:2”, “:3” etc.

If the <PathHdr> element within <KeywordRequest> is omitted, then all instances are returned.

If the Keyboard Handler protocol knows or detects that a particular Keyword is read-only, then the <KeywordResp> must include the “&READONLY” tag.

The <DataFilter> element specifies the optional filter based on data type to use when a request is made. If no filtering is desired, then this element must be omitted from the <KeywordRequest>. Filtering is not guaranteed to work on any platform configuration language that isn’t defined in the UEFI Configuration Namespace Document.

DataFilter.Buffer

HII questions with *EFI_IFR_TYPE_BUFFER* type are treated as this type. This is most commonly represented in ‘C’ as a VOID type, or as a more complex type. Other than the *EFI_IFR_TYPE_BOOLEAN* and *EFI_IFR_TYPE_NUM_x* data types, all of the HII configuration data types are treated as a sequence of data.

DataFilter.Numeric

A sequence of data that must be interpreted as a one, two, four, or eight-byte wide numeric value. For instance, a definition of “Numeric:2” would indicate that the keyword is a two-byte numeric value. If no byte-size designation is specified, then the value may vary in size.

DataFilter.String

HII questions with *EFI_IFR_TYPE_STRING* type are treated as this type.

DataFilter.Boolean

HII questions with *EFI_IFR_TYPE_BOOLEAN* type are treated as this type.

DataFilter.Date

HII questions with *EFI_IFR_TYPE_DATE* type are treated as this type.

DataFilter.Time

HII questions with *EFI_IFR_TYPE_TIME* type are treated as this type.

The <UsageFilter> element defines the optional filter to use based on usage type when a request is made. If no filtering is desired, then this element must be omitted from the <KeywordRequest>.

UsageType.ReadOnly

The data for the keyword cannot be changed. It is intended solely for informational purposes, and can be used to read a setting that may be static or dynamic (e.g. CPU temperature).

UsageType.ReadWrite

The data for the keyword can be changed.

The <KeywordInfoRequest> element allows the callers to request some additional information of the keyword to be returned. <KeywordInfoRequest> element is used when user doesn’t know the information about the keyword and wants to get more information about this keyword. When <KeywordInfoRequest> element is specified with <KeywordRequest>, the <KeywordInfoResp> element will be specified with <KeywordResp> to return the info requested by <KeywordInfoRequest>.

The <KeywordInfoFilter> element is used to specify the additional information that caller wants to know about the keyword. Caller can specify any type of additional information he/she wants to know. When ‘All’ is specified, means all the supported information need to be returned.

The *<DataType>* element specifies the data type of a keyword, can refer to *<DataFilter>* for the detailed info the data types.

The *<ValueAttribute>* element specifies the value attribute of a keyword. Such as the value range of a keyword or the selectable values for a keyword.

<ValueRange> element specifies the variation range of a keyword value. Such as it can be specified for a keyword used in *EFI_IFR_NUMERIC_OP*, *EFI_IFR_STRING_OP* opcode. For *EFI_IFR_NUMERIC_OP* opcode, it specifies the maximum value, minimum value and increment or decrement step. For *EFI_IFR_STRING_OP* opcode, it specifies the maximum length and minimum length of the string can be input.

<ValueOption> element specifies all the (selectable) values and related string representation of these values for a keyword. Such as it can be specified for keyword used in *EFI_IFR_ONE_OF_OP*, *EFI_IFR_ORDERED_LIST_OP* opcode. For *EFI_IFR_ONE_OF_OP*, it specifies the all selectable values and the string representation of the values. The keyword value can be one of them.

For *EFI_IFR_ORDERED_LIST_OP*, it specifies all values and the string representation of the values. The keyword value can be the permutation and combination of these values. And for *EFI_IFR_ORDERED_LIST_OP*, its data type is Buffer, so can return the value type in a *<ValueOption>* to indicate the data stored in the Buffer is numeric as a one, two, four, or eight-byte wide.

The *<Default>* element specifies the default value of a keyword. Only the three standard defaults stores are supported including the standard defaults, the manufacturing defaults and the Safe defaults. If the keyword doesn't have any type of defaults, then there is no default info returned. And if the keyword only has the standard default, then only the standard default information will be returned.

The *<Display>* element specifies the displayed prompt string of this keyword in the UI page.

35.2.1.6.1 An example of some basic keyword-related strings:

<KeywordRequest> to retrieve the current BIOS Vendor name:

```
KEYWORD=BIOSVendor
```

35.2.1.6.2 A possible response might look like:

```
x-UEFI-ns&KEYWORD=BIOSVendor&VALUE=AcmeBIOSCompany
```

If a request was made to retrieve all of the settings for a platform, a user would initiate a call to `KeywordHandleràGetData()` with the `KeywordString` and `NamespaceId` being `NULL`.

35.2.1.6.3 A possible response might look like:

```
x-UEFI-ns&KEYWORD=BIOSVendor&VALUE=AcmeBIOSCompany&x-UEFI-extension
-ACME&KEYWORD=SpecialSettingX&VALUE=3
```

In this case, the string returned tells us that there was one discovered keyword called “BIOSVendor” under the standard UEFI namespace and its value was “AcmeBIOS”. There was also an ACME branded namespace element which was discovered that had a keyword called “SpecialSettingX” whose value was 3.

35.2.1.6.4 An example to get more information of a keyword:

```
KEYWORD=BIOSVendor&KEYWORDINFO=All
```

A possible response might look like:

```
x-UEFI-ns&KEYWORD=BIOSVendor&VALUE=AcmeBIOSCompany&DataType=String&MAX=30&MIN=6&STANDARDDEFAULT=AcmeBIOSCompany&MFGDEFAULT=AcmeBIOSCompany
```

35.2.2 String Types

There are six string types. As can be seen from the BNF, the syntax of all is quite similar. The first three are used in communications between drivers and HII. The last three are used for analogous communication between external applications and HII.

<ConfigRequest> : This string is used by HII to request the current and any alternative configurations from a driver. It consists of routing information and only ampersand separated names.

<ConfigAltResp> : A string in this format is returned by the driver in response to a request to fill in a **<ConfigRequest>** string. The string consists of the current configuration followed by possibly several alternative configurations. The alternative configurations have the *ALTCFG* name / value pair in addition to the usual *GUID*, *NAME*, and *PATH* entries in the routing prefix. The *ALTCFG* value is a Default ID which is used to describe the alternative default configuration.

<ConfigResp> : A string in this format is handed by the HII to the driver to cause the driver to change its configuration. It consists of routing information and name / value pairs which correspond to the questions in the driver's IFR. Only **<ConfigResp>** strings which refer to a driver in question may be handed to that driver. The driver shall reject all others.

<MultiConfigRequest> : A string in this format is handed to HII by an external application in order to request the current and alternate configurations of the system's drivers. The format of this string is a series of **<ConfigRequest>** strings separated by ampersands. The HII's job is to separate the requests and hand them off to the appropriate drivers (as indicated by the routing headers).

<MultiConfigAltResp> : A string in this format is handed back to an external application which has requested the current and alternate configurations of the system's drivers. The format of this string is a series of **<ConfigAltResp>** strings separated by ampersands. The HII creates this string by concatenating the current and alternate configuration strings provided by each driver.

<MultiConfigResp> : A string in this format is handed to the HII in order to update the system's configuration. Analogous to the other "Multi" string formats, its syntax is a series of ampersand separated **<ConfigResp>** strings. Upon receipt, the HII routes the **<ConfigResp>** strings to the corresponding drivers.

35.3 EFI Configuration Keyword Handler Protocol

This section provides a detailed description of the EFI Configuration Keyword Handler Protocol.

35.3.1 EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL

Summary

The *EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL* provides the mechanism to set and get the values associated with a keyword exposed through a x-UEFI- prefixed configuration language namespace.

GUID

```
#define EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL_GUID \
{ 0x0a8badd5, 0x03b8, 0x4d19, \
  {0xb1, 0x28, 0x7b, 0x8f, 0x0e, 0xda, 0xa5, 0x96} }
```

Protocol Interface Structure

```
typedef struct _EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL {
    EFI_CONFIG_KEYWORD_HANDLER_SET_DATA      SetData;
    EFI_CONFIG_KEYWORD_HANDLER_GET_DATA      GetData;
} EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL;
```

Parameters

SetData

Set the data associated with a particular configuration namespace keyword.

GetData

Get the data associated with a particular configuration namespace keyword.

Description

The *EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL* allows other components in the platform (e.g. Browser, Manageability Software, etc.) to retrieve and set configuration settings within the system.

Keywords are text elements which are associated with a particular configuration option within the platform. These keywords are intended to add semantic meaning to the configuration option they are attached to. The text associated for the keyword would be encoded in a UEFI configuration language. These languages are like French or German or Japanese, but are not designed for display purposes for an end-user. Instead each language serves as a namespace for the purposes of grouping and manipulating groups of platform configurations options. See [Working with a UEFI Configuration Language](#) for more information.

NOTE: *Not all configuration options will be associated with a keyword. Associating a keyword with a configuration option is at the discretion of the platform and/or the hardware vendor. For more information about keyword definitions associated with a UEFI namespace, see the [UEFI Keyword Namespace Registry](#) link in the UEFI Link Document.*

35.3.2 EFI_KEYWORD_HANDLER_PROTOCOL.SetData()

Summary

Set the data associated with a particular configuration namespace keyword.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_KEYWORD_HANDLER_SET_DATA) (
    IN EFI_KEYWORD_HANDLER_PROTOCOL      *This,
    IN CONST EFI_STRING                  KeywordString,
    OUT EFI_STRING                        *Progress,
```

(continues on next page)

(continued from previous page)

```
OUT UINT32
);
*ProgressErr
```

Parameters**This**

Pointer to the *EFI_KEYWORD_HANDLER_PROTOCOL* instance.

KeywordString

A null-terminated string in *<MultiKeywordResp>* format.

Progress

On return, points to a character in the *KeywordString*. Points to the string's NULL terminator if the request was successful. Points to the most recent '&' before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) if the request was not successful.

ProgressErr

If during the processing of the *KeywordString* there was a failure, this parameter gives additional information about the possible source of the problem. The various errors are defined in "Related Definitions" below.

Description

This function accepts a *<MultiKeywordResp>* formatted string, finds the associated keyword owners, creates a *<MultiConfigResp>* string from it and forwards it to the *EFI_HII_ROUTING_PROTOCOL.RouteConfig* function.

If there is an issue in resolving the contents of the *KeywordString*, then the function returns an error and also sets the *Progress* and *ProgressErr* with the appropriate information about where the issue occurred and additional data about the nature of the issue.

In the case when *KeywordString* containing multiple keywords, when an *EFI_NOT_FOUND* error is generated during processing the second or later keyword element, the system storage associated with earlier keywords is not modified. All elements of the *KeywordString* must successfully pass all tests for format and access prior to making any modifications to storage.

In the case when *EFI_DEVICE_ERROR* is returned from the processing of a *KeywordString* containing multiple keywords, the state of storage associated with earlier keywords is undefined.

Related Definitions

```
/**
 * Progress Errors
 */
#define KEYWORD_HANDLER_NO_ERROR                0x00000000
#define KEYWORD_HANDLER_NAMESPACE_ID_NOT_FOUND 0x00000001
#define KEYWORD_HANDLER_MALFORMED_STRING       0x00000002
#define KEYWORD_HANDLER_KEYWORD_NOT_FOUND      0x00000004
#define KEYWORD_HANDLER_INCOMPATIBLE_VALUE_DETECTED 0x00000008
#define KEYWORD_HANDLER_ACCESS_NOT_PERMITTED   0x00000010
#define KEYWORD_HANDLER_UNDEFINED_PROCESSING_ERROR 0x80000000
```

The *KEYWORD_HANDLER_x* values describe the error values returned in the *ProgressErr* field.

If no errors were encountered, then *KEYWORD_HANDLER_NO_ERROR* is returned with no bits are set.

If the *<NamespaceId>* specified by the *KeywordString* was not found in any of the registered configuration data, the *KEYWORD_HANDLER_NAMESPACE_ID_NOT_FOUND* bit is set.

If there was an error in the parsing of the *KeywordString*, the *KEYWORD_HANDLER_MALFORMED_STRING* bit is set.

If there was a keyword specified in the *KeywordString* which was not found in any of the registered configuration data, *KEYWORD_HANDLER_KEYWORD_NOT_FOUND* bit is set.

If the value either passed into *KeywordString* (during a *SetData* operation) or the value discovered for the *Keyword* (during a *GetData* operation) did not match what was known to be valid for the defined keyword, the *KEYWORD_HANDLER_INCOMPATIBLE_VALUE_DETECTED* bit is set.

If there was an error as a result of a violation of system policy. For example trying to write a read-only element, the *KEYWORD_HANDLER_ACCESS_NOT_PERMITTED* bit is set.

If there was an undefined type of error in processing the passed in data, the *KEYWORD_HANDLER_UNDEFINED_PROCESSING_ERROR* bit is set.

Status Codes Returned

EFI_SUCCESS	The specified action was completed successfully.
EFI_INVALID_PARAMETER	One or more of the following are TRUE : <i>KeywordString</i> is NULL . Parsing of the <i>KeywordString</i> resulted in an error. See <i>Progress</i> and <i>ProgressErr</i> for more data.
EFI_NOT_FOUND	An element of the <i>KeywordString</i> was not found. See <i>ProgressErr</i> for more data.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated. See <i>ProgressErr</i> for more data.
EFI_ACCESS_DENIED	The action violated system policy. See <i>ProgressErr</i> for more data.
EFI_DEVICE_ERROR	An unexpected system error occurred. See <i>ProgressErr</i> for more data.

35.3.3 EFI_KEYWORD_HANDLER_PROTOCOL.GetData()

Summary

Get the data associated with a particular configuration namespace keyword.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_KEYWORD_HANDLER_GET_DATA) (
    IN EFI_KEYWORD_HANDLER_PROTOCOL      *This,
    IN CONST EFI_STRING                 NamespaceId, OPTIONAL
    IN CONST EFI_STRING                 KeywordString, OPTIONAL
    OUT EFI_STRING                      *Progress,
    OUT UINT32                          *ProgressErr,
    OUT EFI_STRING                      *Results
);
```

Parameters

This

Pointer to the *EFI_KEYWORD_HANDLER_PROTOCOL* instance.

NamespaceId

A null-terminated string containing the platform configuration language to search through in the system. If a **NULL** is passed in, then it is assumed that any platform configuration language with the prefix of “x-UEFI-” are searched.

KeywordString

A null-terminated string in *<MultiKeywordRequest>* format. If a NULL is passed in the *KeywordString* field, all of the known keywords in the system for the *NamespaceId* specified are returned in the Results field.

Progress

On return, points to a character in the *KeywordString*. Points to the string's NULL terminator if the request was successful. Points to the most recent '&' before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) if the request was not successful.

ProgressErr

If during the processing of the *KeywordString* there was a failure, this parameter gives additional information about the possible source of the problem. See the definitions in *SetData()* for valid value definitions.

Results

A null-terminated string in *<MultiKeywordResp>* format is returned which has all the values filled in for the keywords in the *KeywordString*. This is a callee-allocated field, and must be freed by the caller after being used.

Description

This function accepts a *<MultiKeywordRequest>* formatted string, finds the underlying keyword owners, creates a *<MultiConfigRequest>* string from it and forwards it to the *EFI_HII_ROUTING_PROTOCOL.ExtractConfig* function.

If there is an issue in resolving the contents of the *KeywordString*, then the function returns an *EFI_INVALID_PARAMETER* and also set the *Progress* and *ProgressErr* with the appropriate information about where the issue occurred and additional data about the nature of the issue.

In the case when *KeywordString* is **NULL**, or contains multiple keywords, or when *EFI_NOT_FOUND* is generated while processing the keyword elements, the *Results* string contains values returned for all keywords processed prior to the keyword generating the error but no values for the keyword with error or any following keywords.

Status Codes Returned

EFI_SUCCESS	The specified action was completed successfully.
EFI_INVALID_PARAMETER	One or more of the following are TRUE : <i>Progress</i> , <i>ProgressErr</i> , or <i>Results</i> is NULL . Parsing of the <i>KeywordString</i> resulted in an error. See <i>Progress</i> and <i>ProgressErr</i> for more data.
EFI_NOT_FOUND	An element of the <i>KeywordString</i> was not found. See <i>ProgressErr</i> for more data.
EFI_NOT_FOUND	The <i>NamespaceId</i> specified was not found. See <i>ProgressErr</i> for more data.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated. See <i>ProgressErr</i> for more data.
EFI_ACCESS_DENIED	The action violated system policy. See <i>ProgressErr</i> for more data.
EFI_DEVICE_ERROR	An unexpected system error occurred. See <i>ProgressErr</i> for more data.

35.4 EFI HII Configuration Routing Protocol

35.4.1 EFI_HII_CONFIG_ROUTING_PROTOCOL

Summary

The EFI HII Configuration Routing Protocol manages the movement of configuration data from drivers to configuration applications. It then serves as the single point to receive configuration information from configuration applications, routing the results to the appropriate drivers.

GUID

```
#define EFI_HII_CONFIG_ROUTING_PROTOCOL_GUID \
{ 0x587e72d7, 0xcc50, 0x4f79, \
  { 0x82, 0x09, 0xca, 0x29, 0x1f, 0xc1, 0xa1, 0x0f } }
```

Protocol Interface Structure

```
typedef struct {
    EFI_HII_EXTRACT_CONFIG      ExtractConfig;
    EFI_HII_EXPORT_CONFIG       ExportConfig;
    EFI_HII_ROUTE_CONFIG        RouteConfig;
    EFI_HII_BLOCK_TO_CONFIG      BlockToConfig;
    EFI_HII_CONFIG_TO_BLOCK      ConfigToBlock;
    EFI_HII_GET_ALT_CFG          GetAltConfig;
} EFI_HII_CONFIG_ROUTING_PROTOCOL;
```

Related Definitions

None

Parameters

Description

This protocol defines the configuration routing interfaces between external applications and the HII.

There may only be one instance of this protocol in the system.

35.4.2 EFI_HII_CONFIG_ROUTING_PROTOCOL.ExtractConfig()

Summary

This function allows a caller to extract the current configuration for one or more named elements from one or more drivers.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_EXTRACT_CONFIG ) (
    IN CONST EFI_HII_CONFIG_ROUTING_PROTOCOL  *This,
    IN CONST EFI_STRING                        Request,
    OUT EFI_STRING                            *Progress,
    OUT EFI_STRING                            *Results
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ROUTING_PROTOCOL* instance.

Request

A null-terminated string in *<MultiConfigRequest>* format.

Progress

On return, points to a character in the Request string. Points to the string's null terminator if request was successful. Points to the most recent '&' before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) if the request was not successful

Results

A null-terminated string in *<MultiConfigAltResp>* format which has all values filled in for the names in the *Request* string.

Description

This function allows the caller to request the current configuration for one or more named elements from one or more drivers. The resulting string is in the standard HII configuration string format. If Successful *Results* contains an equivalent string with “=” and the values associated with all names added in.

The expected implementation is for each *<ConfigRequest>* substring in the Request, call the HII Configuration Access Protocol *ExtractConfig* function for the driver corresponding to the *<ConfigHdr>* at the start of the *<ConfigRequest>* substring. The request fails if no driver matches the *<ConfigRequest>* substring.

NOTE: *Alternative configuration strings may also be appended to the end of the current configuration string. If they are, they must appear after the current configuration. They must contain the same routing (GUID, NAME, PATH) as the current configuration string. They must have an additional description indicating the type of alternative configuration the string represents, “ALTCFG=<AltCfgId>”. The <AltCfgId> is a reference to a Default ID which stipulates the type of Default being referenced such as EFI_HII_DEFAULT_CLASS_STANDARD.*

As an example, assume that the Request string is:

```
GUID=...&PATH=...&Fred&George&Ron&Neville
```

A result might be:

```
GUID=...&PATH=...&Fred=16&George=16&Ron=12&Neville=11&
GUID=...&PATH=...&ALTCFG=0037&Fred=12&Neville=7
```

NOTE: *For the output Results, the value filled in the names in the Request string with <MultiConfigAltResp> format may change when called multiple times due to some data being of a dynamic nature.*

Status Codes Returned

EFI_SUCCESS	The <i>Results</i> string is filled with the values corresponding to all requested names.
EFI_OUT_OF_RESOURCES	Not enough memory to store the parts of the results that must be stored awaiting possible future protocols.
EFI_NOT_FOUND	Routing data doesn’t match any known driver. Progress set to the “G” in “GUID” of the routing header that doesn’t match. Note: There is no requirement that all routing data be validated before any configuration extraction.
EFI_INVALID_PARAMETER	Illegal syntax. Progress set to most recent “&” before the error or the beginning of the string.
EFI_INVALID_PARAMETER	The ExtractConfig function of the underlying HII Configuration Access Protocol returned EFI_INVALID_PARAMETER. Progress set to most recent “&” before the error or the beginning of the string.
EFI_ACCESS_DENIED	The action violated a system policy.

35.4.3 EFI_HII_CONFIG_ROUTING_PROTOCOL.ExportConfig()

Summary

This function allows the caller to request the current configuration for the entirety of the current HII database and returns the data in a null-terminated string.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_EXPORT_CONFIG ) (
    IN CONST EFI_HII_CONFIG_ROUTING_PROTOCOL  *This,
    OUT EFI_STRING                            *Results
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ROUTING_PROTOCOL* instance.

Results

A null-terminated string in *<MultiConfigAltResp>* format which has all values filled in for the entirety of the current HII database.

Description

This function allows the caller to request the current configuration for all of the current HII database. The results include both the current and alternate configurations as described in *ExtractConfig()* above.

EFI_HII_CONFIG_ACCESS_PROTOCOL.ExtractConfig() interfaces below.

Status Codes Returned

EFI_SUCCESS	The <i>Results</i> string is filled with the values corresponding to all requested names.
EFI_OUT_OF_RESOURCES	Not enough memory to store the parts of the results that must be stored awaiting possible future protocols.
EFI_INVALID_PARAMETER	For example, passing in a NULL for the <i>Results</i> parameter would result in this type of error.

35.4.4 EFI_HII_CONFIG_ROUTING_PROTOCOL.RouteConfig()

Summary

This function processes the results of processing forms and routes it to the appropriate handlers or storage.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_ROUTE_CONFIG ) (
    IN CONST EFI_HII_CONFIG_ROUTING_PROTOCOL  *This,
    IN CONST EFI_STRING                        Configuration,
    OUT EFI_STRING                            *Progress
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ROUTING_PROTOCOL* instance.

Configuration

A null-terminated string in *<MultiConfigResp>* format.

Progress

A pointer to a string filled in with the offset of the most recent ‘&’ before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) or the terminating NULL if all was successful.

Description

This function routes the results of processing forms to the appropriate targets. It scans for *<ConfigHdr>* within the string and passes the header and subsequent body to the driver whose location is described in the *<ConfigHdr>*. Many *<ConfigHdr>* s may appear as a single request.

The expected implementation is to hand off the various *<ConfigResp>* substrings to the Configuration Access Protocol *RouteConfig* routine corresponding to the driver whose routing information is defined by the *<ConfigHdr>* in turn.

Status Codes Returned

EFI_SUCCESS	The results have been distributed or are awaiting distribution.
EFI_OUT_OF_RESOURCES	Not enough memory to store the parts of the results that must be stored awaiting possible future protocols.
EFI_INVALID_PARAMETERS	Passing in a NULL for the <i>Configuration</i> parameter would result in this type of error.
EFI_NOT_FOUND	Target for the specified routing data was not found
EFI_ACCESS_DENIED	The action violated a system policy.

35.4.5 EFI_HII_CONFIG_ROUTING_PROTOCOL.BlockToConfig()

Summary

This helper function is to be called by drivers to map configuration data stored in byte array (“block”) formats such as UEFI Variables into current configuration strings.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_BLOCK_TO_CONFIG ) (
    IN CONST EFI_HII_CONFIG_ROUTING_PROTOCOL    *This,
    IN CONST EFI_STRING                          ConfigRequest,
    IN CONST UINT8                               *Block,
    IN CONST UINTN                              BlockSize,
    OUT EFI_STRING                              *Config,
    OUT EFI_STRING                              *Progress
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ROUTING_PROTOCOL* instance.

ConfigRequest

A null-terminated string in *<ConfigRequest>* format.

Block

Array of bytes defining the block's configuration.

BlockSize

Length in bytes of *Block*.

Config

Filled-in configuration string. String allocated by the function. Returned only if call is successful. The null-terminated string will be in *<ConfigResp>* format

Progress

A pointer to a string filled in with the offset of the most recent ‘&’ before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) or the terminating NULL if all was successful.

Description

This function extracts the current configuration from a block of bytes. To do so, it requires that the *ConfigRequest* string consists of a list of *<BlockName>* formatted names. It uses the offset in the name to determine the index into the Block to start the extraction and the width of each name to determine the number of bytes to extract. These are mapped to a string using the equivalent of the C “%x” format (with optional leading spaces).

The call fails if, for any (offset, width) pair in *ConfigRequest*, offset+value $\geq$ *BlockSize*.

Status Codes Returned

EFI_SUCCESS	The request succeeded. Progress points to the null terminator at the end of the <i>ConfigRequest</i> string.
EFI_OUT_OF_RESOURCES	Not enough memory to allocate <i>Config</i> . Progress points to the first character of <i>ConfigRequest</i> .
EFI_INVALID_PARAMETERS	Passing in a NULL for the <i>ConfigRequest</i> or <i>Block</i> parameter would result in this type of error. Progress points to the first character of <i>ConfigRequest</i> .
EFI_NOT_FOUND	Target for the specified routing data was not found. <i>Progress</i> points to the “G” in “GUID” of the errant routing data.
EFI_DEVICE_ERROR	Block not large enough. <i>Progress</i> undefined.
EFI_INVALID_PARAMETER	Encountered non <i><BlockName></i> formatted string. Block is left updated and Progress points at the ‘&’ preceding the first non- <i><BlockName></i> .

35.4.6 EFI_HII_CONFIG_ROUTING_PROTOCOL.ConfigToBlock()**Summary**

This helper function is to be called by drivers to map configuration strings to configurations stored in byte array (“block”) formats such as UEFI Variables.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_CONFIG_TO_BLOCK ) (
    IN    CONST EFI_HII_CONFIG_ROUTING_PROTOCOL    *This,
    IN    CONST EFI_STRING                          *ConfigResp,
    IN OUT CONST UINT8                             *Block,
```

(continues on next page)

(continued from previous page)

```
IN OUT UINTN          *BlockSize,
OUT   EFI_STRING      *Progress
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ROUTING_PROTOCOL* instance.

ConfigResp

A null-terminated string in *<ConfigResp>* format.

Block

A possibly null array of bytes representing the current block. Only bytes referenced in the *ConfigResp* string in the block are modified. If this parameter is null or if the **BlockSize* parameter is (on input) shorter than required by the *Configuration* string, only the *BlockSize* parameter is updated and an appropriate status (see below) is returned.

BlockSize

The length of the *Block* in units of *UINT8*. On input, this is the size of the *Block*. On output, if successful, contains the largest index of the modified byte in the *Block*, or the required buffer size if the *Block* is not large enough.

Progress

On return, points to an element of the *ConfigResp* string filled in with the offset of the most recent ‘&’ before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) or the terminating NULL if all was successful.

Description

This function maps a configuration containing a series of *<BlockConfig>* formatted name value pairs in *ConfigResp* into a *Block* so it may be stored in a linear mapped storage such as a UEFI Variable. If present, the function skips *GUID*, *NAME*, and *PATH* in *<ConfigResp>*. It stops when it finds a non- *<BlockConfig>* name / value pair (after skipping the routing header) or when it reaches the end of the string.

Example

Assume an existing block containing:

```
00 01 02 03 04 05
```

And the *ConfigResp* string is:

```
OFFSET=3WIDTH=1&VALUE=7&OFFSET=0&WIDTH=2&VALUE=AA55
```

The results are

```
55 AA 02 07 04 05
```

Status Codes Returned

EFI_SUCCESS	The request succeeded. Progress points to the null terminator at the end of the <i>ConfigResp</i> string.
EFI_OUT_OF_RESOURCES	Not enough memory to allocate <i>Config</i> . Progress points to the first character of <i>ConfigResp</i> .

continues on next page

Table 35.7 – continued from previous page

EFI_INVALID_PARAMETER	Passing in a NULL for the <i>ConfigResp</i> or <i>Block</i> parameter would result in this type of error. <i>Progress</i> points to the first character of <i>ConfigResp</i> .
EFI_NOT_FOUND	Target for the specified routing data was not found. <i>Progress</i> points to the “G” in “GUID” of the errant routing data.
EFI_BUFFER_TOO_SMALL	Block not large enough. <i>Progress</i> undefined. <i>BlockSize</i> is updated with the required buffer size.
EFI_INVALID_PARAMETER	Encountered non <BlockName> formatted name / value pair. <i>Block</i> is left updated and <i>Progress</i> points at the ‘&’ preceding the first non-<BlockName>.

35.4.7 EFI_HII_CONFIG_ROUTING_PROTOCOL.GetAltCfg()

Summary

This helper function is to be called by drivers to extract portions of a larger configuration string.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_GET_ALT_CFG ) (
    IN  CONST EFI_HII_CONFIG_ROUTING_PROTOCOL    *This,
    IN  CONST EFI_STRING                        ConfigResp,
    IN  CONST EFI_GUID                         *Guid,
    IN  CONST EFI_STRING                       Name,
    IN  CONST EFI_DEVICE_PATH_PROTOCOL         *DevicePath,
    IN  CONST EFI_STRING                       AltCfgId,
    OUT EFI_STRING                             *AltCfgResp
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ROUTING_PROTOCOL* instance.

ConfigResp

A null-terminated string in <*ConfigAltResp*> format.

Guid

A pointer to the GUID value to search for in the routing portion of the *ConfigResp* string when retrieving the requested data. If *Guid* is NULL, then all GUID values will be searched for.

Name

A pointer to the NAME value to search for in the routing portion of the *ConfigResp* string when retrieving the requested data. If *Name* is NULL, then all *Name* values will be searched for.

DevicePath

A pointer to the PATH value to search for in the routing portion of the *ConfigResp* string when retrieving the requested data. If *DevicePath* is NULL, then all *DevicePath* values will be searched for.

AltCfgId

A pointer to the ALTCFG value to search for in the routing portion of the *ConfigResp* string when retrieving the requested data. If this parameter is NULL, then the current setting will be retrieved.

AltCfgResp

A pointer to a buffer which will be allocated by the function which contains the retrieved string as requested. This

buffer is only allocated if the call was successful. The null-terminated string will be in *<ConfigResp>* format.

Description

This function retrieves the requested portion of the configuration string from a larger configuration string. This function will use the *Guid*, *Name*, and *DevicePath* parameters to find the appropriate section of the *ConfigResp* string. Upon finding this portion of the string, it will use the *AltCfgId* parameter to find the appropriate instance of data in the *ConfigResp* string. Once found, the found data will be copied to a buffer which is allocated by the function so that it can be returned to the caller. The caller is responsible for freeing this allocated buffer.

Status Codes Returned

EFI_SUCCESS	The request succeeded. The requested data was extracted and placed in the newly allocated <i>AltCfgResp</i> buffer.
EFI_OUT_OF_RESOURCES	Not enough memory to allocate <i>AltCfgResp</i> .
EFI_INVALID_PARAMETER	Passing in a NULL for the <i>ConfigResp</i> or <i>AltCfgResp</i> would result in this type of error.

35.5 EFI HII Configuration Access Protocol

35.5.1 EFI_HII_CONFIG_ACCESS_PROTOCOL

Summary

The EFI HII configuration routing protocol invokes this type of protocol when it needs to forward requests to a driver's configuration handler. This protocol is published by drivers providing and receiving configuration data from HII. The *ExtractConfig()* and *RouteConfig()* functions are typically invoked by the driver which implements the HII Configuration Routing Protocol. The *Callback()* function is typically invoked by the Forms Browser.

If the protocol functions modify active form set, they must not change layout and size of the existing variable stores. The forms browser processes updated IFR package in accordance with the following rules:

1. If active form set no longer exists, the behavior is browser specific. The browser identifies form set using a combination of the form set GUID and device path associated with the package list containing the form set.
2. If form set update has been initiated by the *Callback()* function, the browser executes action requested by the function. See *EFI_HII_CONFIG_ACCESS_PROTOCOL.Callback()* section for additional details regarding browser action requests.

NOTE: *If browser action implies saving of the modified questions values, the browser will use uncommitted data associated with the old form set instance. The HII Configuration Access implementation is responsible for properly handling such requests.*

3. The browser performs standard processing steps that are performed on a form set prior to displaying it (including reading question values and generating *EFI_BROWSER_ACTION_FORM_OPEN* and *EFI_BROWSER_ACTION_FORM_RETRIEVE* callbacks).
4. If there is an uncommitted browser data associated with an active form set, the browser applies it, matching variable stores by their identifiers. If variable store no longer exists, the uncommitted data for this store is discarded.

NOTE: *Changing layout or size of the existing variable stores during form set update is not allowed and can lead to unpredictable results.*

5. The browser applies prior browsing history, matching forms by their identifiers. If a form saved in the browsing history no longer exists, the behavior is browser-specific.

6. If all forms in the browsing history have been matched, the browser sets selection on a question that was active prior to the form set update, matching question by its identifier. If question does not exist, the first question on the form is selected.

GUID

```
#define EFI_HII_CONFIG_ACCESS_PROTOCOL_GUID \
{ 0x330d4706, 0xf2a0, 0x4e4f, \
{0xa3, 0x69, 0xb6, 0x6f, 0xa8, 0xd5, 0x43, 0x85}}
```

Protocol Interface Structure

```
typedef struct {
    EFI_HII_ACCESS_EXTRACT_CONFIG    ExtractConfig;
    EFI_HII_ACCESS_ROUTE_CONFIG      RouteConfig;
    EFI_HII_ACCESS_FORM_CALLBACK     Callback;
} EFI_HII_CONFIG_ACCESS_PROTOCOL;
```

Related Definitions

None

Parameters

ExtractConfig

This function breaks apart the request strings routing them to the appropriate drivers. This function is analogous to the similarly named function in the HII Routing Protocol.

RouteConfig

This function breaks apart the results strings and returns configuration information as specified by the request.

Callback

This function is called from the configuration browser to communicate certain activities that were initiated by a user.

Description

This protocol provides a callable interface between the HII and drivers. Only drivers which provide IFR data to HII are required to publish this protocol.

35.5.2 EFI_HII_CONFIG_ACCESS_PROTOCOL.ExtractConfig()

Summary

This function allows a caller to extract the current configuration for one or more named elements from the target driver.

Prototype

```
typedef
EFI_STATUS
(EFI_API * EFI_HII_ACCESS_EXTRACT_CONFIG ) (
    IN CONST EFI_HII_CONFIG_ACCESS_PROTOCOL    *This,
    IN CONST EFI_STRING                        Request,
    OUT EFI_STRING                             *Progress,
    OUT EFI_STRING                             *Results
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ACCESS_PROTOCOL*.

Request

A null-terminated string in *<ConfigRequest>* format. Note that this includes the routing information as well as the configurable name / value pairs. It is invalid for this string to be in *<MultiConfigRequest>* format.

If a **NULL** is passed in for the *Request* field, all of the settings being abstracted by this function will be returned in the *Results* field. In addition, if a *ConfigHdr* is passed in with no request elements, all of the settings being abstracted for that particular *ConfigHdr* reference will be returned in the *Results* Field.

Progress

On return, points to a character in the *Request* string. Points to the string's null terminator if request was successful. Points to the most recent '&' before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) if the request was not successful

Results

A null-terminated string in *<MultiConfigAltResp>* format which has all values filled in for the names in the *Request* string. String to be allocated by the called function.

Description

This function allows the caller to request the current configuration for one or more named elements. The resulting string is in *<ConfigAltResp>* format.

In order to support forms processors other than a Forms Browser, the configuration returned by this function must not depend on context in which the function is used. In particular, it must not depend on the current state of the Forms Browser (including any uncommitted state information) and actions performed by the driver callbacks invoked prior to the *ExtractConfig* call. See: [numref:Connected-forms-browser-processor](#) for more details.

Any and all alternative configuration strings shall also be appended to the end of the current configuration string. If they are, they must appear after the current configuration. They must contain the same routing (GUID, NAME, PATH) as the current configuration string. They must have an additional description indicating the type of alternative configuration the string represents, " *ALTCFG=<AltCfgId>* ". The *<AltCfgId>* is a reference to a Default ID which stipulates the type of Default being referenced such as *EFI_HII_DEFAULT_CLASS_STANDARD*.

As an example, assume that the *Request* string is:

```
GUID=...&PATH=...&Fred&George&Ron&Neville
```

A result might be:

```
GUID=...&PATH=...&Fred=16&George=16&Ron=12&Neville=11&GUID=...&PATH=...&ALTCFG=0037&
Fred=12&Neville=7
```

This function allows the caller to request the current configuration for one or more named elements. The resulting string is in *<ConfigAltResp>* format.

Any and all alternative configuration strings shall also be appended to the end of the current configuration string. If they are, they must appear after the current configuration. They must contain the same routing (GUID, NAME, PATH) as the current configuration string. They must have an additional description indicating the type of alternative configuration the string represents, " *ALTCFG=<AltCfgId>* ". The *<AltCfgId>* is a reference to a Default ID which stipulates the type of Default being referenced such as *EFI_HII_DEFAULT_CLASS_STANDARD*.

As an example, assume that the *Request* string is:

```
GUID=...&PATH=...&Fred&George&Ron&Neville
```

A result might be:

```
GUID=...&PATH=...&Fred=16&George=16&Ron=12&Neville=11&
GUID=...&PATH=...&ALTCFG=0037&Fred=12&Neville=7
```

NOTE: For the output *Results*, the value filled in the names in the *Request* string with <ConfigAltResp> format may change when called multiple times due to some data being of a dynamic nature.

Status Codes Returned

EFI_SUCCESS	The <i>Results</i> string is filled with the values corresponding to all requested names.
EFI_OUT_OF_RESOURCES	Not enough memory to store the parts of the results that must be stored awaiting possible future protocols.
EFI_NOT_FOUND	A configuration element matching the routing data is not found. Progress set to the first character in the routing header.
EFI_INVALID_PARAMETER	Illegal syntax. Progress set to most recent “&” before the error or the beginning of the string.
EFI_INVALID_PARAMETER	Unknown name. <i>Progress</i> points to the “&” before the name in question.
EFI_INVALID_PARAMETER	If <i>Results</i> or <i>Progress</i> is NULL .
EFI_ACCESS_DENIED	The action violated a system policy.
EFI_DEVICE_ERROR	Failed to extract the current configuration for one or more named elements.

35.5.3 EFI_HII_CONFIG_ACCESS_PROTOCOL.RouteConfig()

Summary

This function processes the results of changes in configuration for the driver that published this protocol.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_CONFIG_ACCESS_ROUTE_CONFIG ) (
    IN CONST EFI_HII_CONFIG_ACCESS_PROTOCOL    *This,
    IN CONST EFI_STRING                        Configuration,
    OUT EFI_STRING                            *Progress
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ACCESS_PROTOCOL*.

Configuration

A null-terminated string in <ConfigResp> format.

Progress

A pointer to a string filled in with the offset of the most recent ‘&’ before the first failing name / value pair (or the beginning of the string if the failure is in the first name / value pair) or the terminating **NULL** if all was successful.

Description

This function applies changes in a driver’s configuration. Input is a *Configuration*, which has the routing data for this driver followed by name / value configuration pairs. The driver must apply those pairs to its configurable storage.

In order to support forms processors other than a Forms Browser, the way in which configuration data is applied must not depend on context in which the function is used. In particular, it must not depend on the current state of the Forms Browser (including any uncommitted state information) and actions performed by the driver callbacks invoked prior to the *RouteConfig* call. *Connected Forms Browser/Processor* provides additional details regarding forms browser/processor.

If the driver's configuration is stored in a linear block of data and the driver's name / value pairs are in *<BlockConfig>* format, it may use the *ConfigToBlock* helper function (above) to simplify the job.

Status Codes Returned

EFI_SUCCESS	The results have been distributed or are awaiting distribution.
EFI_OUT_OF_RESOURCES	Not enough memory to store the parts of the results that must be stored awaiting possible future protocols.
EFI_INVALID_PARAMETER	If Configuration or Progress is NULL.
EFI_NOT_FOUND	Target for the specified routing data was not found
EFI_ACCESS_DENIED	The action violated a system policy.

35.5.4 EFI_HII_CONFIG_ACCESS_PROTOCOL.CallBack()

Summary

This function is called to provide results data to the driver.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_HII_CONFIG_ACCESS_PROTOCOL) (
    IN    CONST EFI_HII_CONFIG_ACCESS_PROTOCOL    *This,
    IN    EFI_BROWSER_ACTION                      Action,
    IN    EFI_QUESTION_ID                        QuestionId,
    IN    UINT8                                  Type
    IN OUT EFI_IFR_TYPE_VALUE                    *Value,
    OUT    EFI_BROWSER_ACTION_REQUEST             *ActionRequest,
);
```

Parameters

This

Points to the *EFI_HII_CONFIG_ACCESS_PROTOCOL*.

Action

Specifies the type of action taken by the browser. See *EFI_BROWSER_ACTION_x* in “Related Definitions” below.

QuestionId

A unique value which is sent to the original exporting driver so that it can identify the type of data to expect. The format of the data tends to vary based on the opcode that generated the callback.

Type

The type of value for the question. See *EFI_IFR_TYPE_x* in *EFI_IFR_ONE_OF_OPTION*.

Value

A pointer to the data being sent to the original exporting driver. The type is specified by *Type*. Type *EFI_IFR_TYPE_VALUE* is defined in *EFI_IFR_ONE_OF_OPTION*.

ActionRequest

On return, points to the action requested by the callback function. Type *EFI_BROWSER_ACTION_REQUEST* is specified in *SendForm()* in the Form Browser Protocol.

Description

This function is called by the forms browser in response to a user action on a question which has the *EFI_IFR_FLAG_CALLBACK* bit set in the *EFI_IFR_QUESTION_HEADER*. The user action is specified by *Action*. Depending on the action, the browser may also pass the question value using *Type* and *Value*. Upon return, the callback function may specify the desired browser action.

The browser maintains uncommitted browser data (modified and unsaved question values) across Callback function boundaries. Callback function may change unsaved question values using one of the following methods:

- Current question's value may be changed by updating the *Value* parameter.
- Values of other questions from the active formset can be changed using *EFI_FORM_BROWSER2_PROTOCOL.BrowserCallback()* interface.

NOTE: *Modification of the question values by the Callback function without notifying the browser using one of the above mentioned methods can lead to unpredictable browser behavior.*

Callback function may request configuration update from the browser by returning an appropriate *ActionRequest*.

In order to save uncommitted data, driver should return one of the *_SUBMIT* actions or *_APPLY* action. The browser will then write all modified question values (in case of the *_SUBMIT* actions) or modified question values from an active form (in case of the *_APPLY* action) to storage using *RouteConfig()* function. This will include questions modified prior to an invocation of the *Callback()* function as well as questions modified by the *Callback()* function.

The behavior of the *ExtractConfig* and *RouteConfig* functions must not depend on the actions performed by this function.

Callback functions should return *EFI_UNSUPPORTED* for all values of *Action* that they do not support.

Related Definitions

```
typedef UINTN EFI_BROWSER_ACTION;

#define EFI_BROWSER_ACTION_CHANGING          0
#define EFI_BROWSER_ACTION_CHANGED          1
#define EFI_BROWSER_ACTION_RETRIEVE         2
#define EFI_BROWSER_ACTION_FORM_OPEN        3
#define EFI_BROWSER_ACTION_FORM_CLOSE       4
#define EFI_BROWSER_ACTION_SUBMITTED        5
#define EFI_BROWSER_ACTION_DEFAULT_STANDARD 0x1000
#define EFI_BROWSER_ACTION_DEFAULT_MANUFACTURING 0x1001
#define EFI_BROWSER_ACTION_DEFAULT_SAFE     0x1002
#define EFI_BROWSER_ACTION_DEFAULT_PLATFORM 0x2000
#define EFI_BROWSER_ACTION_DEFAULT_HARDWARE 0x3000
#define EFI_BROWSER_ACTION_DEFAULT_FIRMWARE 0x4000
```

The following table describes the behavior of the callback for each question type.

Table 35.11: Callback Behavior

Question Type	Type	Action
Action Button	<i>EFI_IFR_TYPE_ACTION</i>	No special behavior. If the short form of the opcode is used, then the value will be a string identifier of zero.
Checkbox	<i>EFI_IFR_TYPE_BOOLEAN</i>	No special behavior

continues on next page

Table 35.11 – continued from previous page

Cross-Reference		
	<i>EFI_IFR_TYPE_REF</i> <i>EFI_IFR_TYPE_UNDEFINED</i>	CHANGING: If <i>EFI_UNSUPPORTED</i> or <i>EFI_SUCCESS</i> , the updated cross-reference is taken. Any other error the cross-reference will not be taken. CHANGED: Never called. RETRIEVE: Called before displaying the cross-reference. Error codes ignored. The Ref field of the Value parameter is initialized with the REF question's value prior to CHANGING and RETRIEVE.
Date	<i>EFI_IFR_TYPE_DATE</i>	No special behavior No special behavior.
Numeric, One-Of	<i>EFI_IFR_TYPE_NUM_SIZE_8</i> , <i>EFI_IFR_TYPE_NUM_SIZE_16</i> , <i>EFI_IFR_TYPE_NUM_SIZE_32</i> , <i>EFI_IFR_TYPE_NUM_SIZE_64</i>	
Ordered-List	<i>EFI_IFR_TYPE_BUFFER</i>	No special behavior
String, Pass- word	<i>EFI_IFR_TYPE_STRING</i>	No special behavior.
Time	<i>EFI_IFR_TYPE_DATE</i>	No special behavior.

The value *EFI_BROWSER_ACTION_CHANGING* is called before the browser changes the value in the display (for questions which have a value) or takes an action (in the case of an action button or cross-reference). If the callback returns *EFI_UNSUPPORTED*, then the browser will use the value passed to *Callback()* and ignore the value returned by *Callback()*. If the callback returns *EFI_SUCCESS*, then the browser will use the value returned by *Callback()*. If any other error is returned, then the browser will not update the current question value. *ActionRequest* is used. The *Value* represents the updated value. The changes here should not be finalized until the user submits the results.

The value *EFI_BROWSER_ACTION_CHANGED* is called after the browser has changed its internal copy of the question value and displayed it (if appropriate). For action buttons, this is called after the value has been processed. For cross-references, this is never called. Errors returned are ignored. *ActionRequest* is used. The changes here should not be finalized until the user submits the results.

The value *EFI_BROWSER_ACTION_RETRIEVE* is called after the browser has read the current question value, but before it has been displayed. If the callback returns *EFI_UNSUPPORTED* or any other error then the original value is used. If *EFI_SUCCESS* is returned, then the updated value is used.

The value *EFI_BROWSER_ACTION_FORM_OPEN* is called for each question on a form prior to its value being retrieved or displayed. If a question appears on more than one form, and the Forms Browser supports more than one form being active simultaneously, this may be called more than once, even prior to any *EFI_BROWSER_ACTION_FORM_CLOSE* callback.

NOTE: *EFI_FORM_BROWSER2_PROTOCOL.BrowserCallback()* cannot be used with this browser action because question values have not been retrieved yet.

The value *EFI_BROWSER_ACTION_FORM_CLOSE* is called for each question on a form after the processing of any submit actions for that form. If a question appears on more than one form, and the Forms Processor supports more than one form being active simultaneously, this will be called more than once.

The value *EFI_BROWSER_ACTION_SUBMITTED* is called after Browser submits the modified question value. *ActionRequest* is ignored.

When *Action* specifies one of the “default” actions, such as *EFI_BROWSER_ACTION_DEFAULT_STANDARD*, etc. it

indicates that the Forms Processor is attempting to retrieve the default value for the specified question. The proposed default value is passed in using *Type* and *Value* and reflects the value which the Forms Processor was able to select based on the lower-priority defaulting methods (see *Defaults*). If the function returns *EFI_SUCCESS*, then the updated value will be used. If the function does not have an updated default value for the specified question or specified default store, or does not provide any support for the actions, it should return *EFI_UNSUPPORTED*, and the returned value will be ignored.

The *DEFAULT_PLATFORM*, *DEFAULT_HARDWARE* and *DEFAULT_FIRMWARE* represent ranges of 4096 (0x1000) possible default store identifiers. The *DEFAULT_STANDARD* represents the range of 4096 possible action values reserved for UEFI-defined default store identifiers. See *Defaults* for more information on defaults.

```
typedef UINTN EFI_BROWSER_ACTION_REQUEST;

#define EFI_BROWSER_ACTION_REQUEST_NONE          0
#define EFI_BROWSER_ACTION_REQUEST_RESET        1
#define EFI_BROWSER_ACTION_REQUEST_SUBMIT       2
#define EFI_BROWSER_ACTION_REQUEST_EXIT         3
#define EFI_BROWSER_ACTION_REQUEST_FORM_SUBMIT_EXIT 4
#define EFI_BROWSER_ACTION_REQUEST_FORM_DISCARD_EXIT 5
#define EFI_BROWSER_ACTION_REQUEST_FORM_APPLY   6
#define EFI_BROWSER_ACTION_REQUEST_FORM_DISCARD 7
#define EFI_BROWSER_ACTION_REQUEST_RECONNECT    8
#define EFI_BROWSER_ACTION_REQUEST_QUESTION_APPLY 9
```

If the callback function returns with the *ActionRequest* set to *_NONE*, then the Forms Browser will take no special behavior.

If the callback function returns with the *ActionRequest* set to *_RESET*, then the Forms Browser will exit and request the platform to reset.

If the callback function returns with the *ActionRequest* set to *_SUBMIT*, then the Forms Browser will save all modified question values to storage and exit.

If the callback function returns with the *ActionRequest* set to *_EXIT*, then the Forms Browser will discard all modified question values and exit.

If the callback function returns with the *ActionRequest* set to *_FORM_SUBMIT_EXIT*, then the Forms Browser will write all modified question values on the selected form to storage and then exit the selected form.

If the callback function returns with the *ActionRequest* set to *_FORM_DISCARD_EXIT*, then the Forms Browser will discard the modified question values on the selected form and then exit the selected form.

If the callback function returns with the *ActionRequest* set to *_FORM_APPLY*, then the Forms Browser will write all modified current question values on the selected form to storage.

If the callback function returns with the *ActionRequest* set to *_FORM_DISCARD*, then the Forms Browser will discard the current question values on the selected form and replace them with the original question values.

If the callback function returns with the *ActionRequest* set to *_RECONNECT*, a hardware and/or software configuration change was performed by the user, and the controller needs to be reconnected for the driver to recognize the change. Upon the user exiting the formset or the browser, the Forms Browser is required to call the EFI Boot Service *DisconnectController()* followed by the EFI Boot Service *ConnectController()* to reconnect the controller. The controller handle passed to *DisconnectController()* and *ConnectController()* is the handle on which this *EFI_HII_CONFIG_ACCESS_PROTOCOL* is installed.

If the callback function returns with the *ActionRequest* set to *_QUESTION_APPLY*, then the Forms Browser will write the current modified question value on the selected form to storage.

Status Codes Returned

EFI_SUCCESS	The callback successfully handled the action.
EFI_OUT_OF_RESOURCES	Not enough storage is available to hold the variable and its data.
EFI_DEVICE_ERROR	The variable could not be saved.
EFI_UNSUPPORTED	The specified Action is not supported by the callback.

35.6 Form Browser Protocol

The *EFI_FORM_BROWSER2_PROTOCOL* is the interface to call for drivers to leverage the EFI configuration driver interface.

35.6.1 EFI_FORM_BROWSER2_PROTOCOL

Summary

The *EFI_FORM_BROWSER2_PROTOCOL* is the interface to the UEFI configuration driver. This interface will allow the caller to direct the configuration driver to use either the HII database or use the passed-in packet of data.

GUID

```
#define EFI_FORM_BROWSER2_PROTOCOL_GUID \
{ 0xb9d4c360, 0xbcfb, 0x4f9b, \
  { 0x92, 0x98, 0x53, 0xc1, 0x36, 0x98, 0x22, 0x58 } }
```

Protocol Interface Structure

```
typedef struct _EFI_FORM_BROWSER2_PROTOCOL {
    EFI_SEND_FORM2                SendForm;
    EFI_BROWSER_CALLBACK2         BrowserCallback;
} EFI_FORM_BROWSER2_PROTOCOL;
```

Parameters

SendForm

Browse the specified configuration forms. See the *SendForm()* function description.

BrowserCallback

Routine used to expose internal configuration state of the browser. This is primarily used by callback handler routines which were called by the browser and in-turn need to get additional information from the browser itself. See the *BrowserCallback()* function description.

Description

This protocol is the interface to call for drivers to leverage the EFI configuration driver interface.

35.6.2 EFI_FORM_BROWSER2_PROTOCOL.SendForm()

Summary

Initialize the browser to display the specified configuration forms.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_SEND_FORM2) (
    IN CONST EFI_FORM_BROWSER2_PROTOCOL    *This,
    IN EFI_HII_HANDLE                      *Handles,
    IN UINTN                               HandleCount,
    IN CONST EFI_GUID                      *FormsetGuid, OPTIONAL
    IN EFI_FORM_ID                          FormId, OPTIONAL
    IN CONST EFI_SCREEN_DESCRIPTOR         *ScreenDimensions, OPTIONAL
    OUT EFI_BROWSER_ACTION_REQUEST         *ActionRequest OPTIONAL
);
```

Parameters

This

A pointer to the *EFI_FORM_BROWSER2_PROTOCOL* instance.

Handles

A pointer to an array of HII handles to display. This value should correspond to the value of the HII form package that is required to be displayed. Type *EFI_HII_HANDLE* is defined in *EFI_HII_DATABASE_PROTOCOL.NewPackageList()* in *Package Lists and Package Headers*.

HandleCount

The number of handles in the array specified by *Handle*.

FormsetGuid

This field points to the *EFI_GUID* which must match the *Guid* field or one of the elements of the *ClassId* field in the *EFI_IFR_FORM_SET* op-code. If *FormsetGuid* is *NULL*, then this function will display the form set class *EFI_HII_PLATFORM_SETUP_FORMSET_GUID*.

FormId

This field specifies the identifier of the form within the form set to render as the first displayable page. If this field has a value of 0x0000, then the Forms Browser will render the first enabled form in the form set.

ScreenDimensions

Points to recommended form dimensions, including any non-content area, in characters. Type *EFI_SCREEN_DESCRIPTOR* is defined in “Related Definitions” below.

ActionRequested

Points to the action recommended by the form.

Description

This function is the primary interface to the Forms Browser. The Forms Browser displays the forms specified by *FormsetGuid* and *FormId* from all of HII handles specified by *Handles*. If more than one form can be displayed, the Forms Browser will provide some means for the user to navigate between the forms in addition to that provided by cross-references in the forms themselves.

If *ScreenDimensions* is non-*NULL*, then it points to a recommended display size for the form. If browsing in text mode, then these are recommended character positions. If browsing in graphics mode, then these values are converted to pixel locations using the standard font size (8 pixels per horizontal character cell and 19 pixels per vertical character cell).

If `ScreenDimensions` is `NULL` the browser may choose the size based on platform policy. The browser may choose to ignore the size based on platform policy.

If `ActionRequested` is non-`NULL`, then upon return, it points to an enumerated value (see `EFI_BROWSER_ACTION_x` in “Related Definitions” below) which describes the action requested by the user. If set to `EFI_BROWSER_ACTION_NONE`, then no specific action was requested by the form. If set to `EFI_BROWSER_ACTION_RESET`, then the form requested that the platform be reset. The browser may, based on platform policy, ignore such action requests.

If `FormsetGuid` is set to `EFI_HII_PLATFORM_SETUP_FORMSET_GUID`, it indicates that the form set contains forms designed to be used for platform configuration. If `FormsetGuid` is set to `EFI_HII_DRIVER_HEALTH_FORMSET_GUID`, it indicates that the form set contains forms designed to be used for support of the Driver Health Protocol (*EFI Driver Health Protocol*). If `FormsetGuid` is set to `EFI_HII_USER_CREDENTIAL_FORMSET_GUID`, it indicates that the form set contains forms designed to be used for support of the User Credential Protocol (*Credential Provider Protocols*). If `FormsetGuid` is set to `EFI_HII_REST_STYLE_FORMSET_GUID`, it indicates that the form set contains forms designed to be used for support configuration of REST architectural style (see Section 29.7). Other values may be used for other applications.

Related Definitions

```
//*****
// EFI_SCREEN_DESCRIPTOR
//*****
typedef struct {
    UINTN      LeftColumn;
    UINTN      RightColumn;
    UINTN      TopRow;
    UINTN      BottomRow;
} EFI_SCREEN_DESCRIPTOR;
```

LeftColumn

Value that designates the text column where the browser window will begin from the left-hand side of the screen

RightColumn

Value that designates the text column where the browser window will end on the right-hand side of the screen.

TopRow

Value that designates the text row from the top of the screen where the browser window will start.

BottomRow

Value that designates the text row from the bottom of the screen where the browser window will end.

```
typedef UINTN EFI_BROWSER_ACTION_REQUEST;

#define EFI_BROWSER_ACTION_REQUEST_NONE      0
#define EFI_BROWSER_ACTION_REQUEST_RESET    1
#define EFI_BROWSER_ACTION_REQUEST_SUBMIT    2
#define EFI_BROWSER_ACTION_REQUEST_EXIT     3
```

The value `EFI_BROWSER_ACTION_REQUEST_NONE` indicates that no specific caller action is required. The value `EFI_BROWSER_ACTION_REQUEST_RESET` indicates that the caller requested a platform reset. The value `EFI_BROWSER_ACTION_REQUEST_SUBMIT` indicates that a callback requested that the browser submit all values and exit. The value `EFI_BROWSER_ACTION_REQUEST_EXIT` indicates that a callback requested that the browser exit without saving all values.

```
#define EFI_HII_PLATFORM_SETUP_FORMSET_GUID \
{ 0x93039971, 0x8545, 0x4b04, \
```

(continues on next page)

(continued from previous page)

```

{ 0xb4, 0x5e, 0x32, 0xeb, 0x83, 0x26, 0x04, 0x0e } }

#define EFI_HII_DRIVER_HEALTH_FORMSET_GUID \
{ 0xf22fc20c, 0x8cf4, 0x45eb, \
  { 0x8e, 0x06, 0xad, 0x4e, 0x50, 0xb9, 0x5d, 0xd3 } }

#define EFI_HII_USER_CREDENTIAL_FORMSET_GUID \
{ 0x337f4407, 0x5aee, 0x4b83, \
  { 0xb2, 0xa7, 0x4e, 0xad, 0xca, 0x30, 0x88, 0xcd } }

#define EFI_HII_REST_STYLE_FORMSET_GUID \
{ 0x790217bd, 0xbecf, 0x485b, \
  { 0x91, 0x70, 0x5f, 0xf7, 0x11, 0x31, 0x8b, 0x27 } }

```

Status Codes Returned

EFI_SUCCESS	The function completed successfully
EFI_NOT_FOUND	No valid forms could be found to display.
EFI_INVALID_PARAMETER	One of the parameters has an invalid value.

35.6.3 EFI_FORM_BROWSER2_PROTOCOL.BrowserCallback()**Summary**

This function is called by a callback handler to retrieve uncommitted state data from the browser.

Prototype

```

EFI_STATUS
(EFIAPI *EFI_BROWSER_CALLBACK2 ) (
    IN      CONST EFI_FORM_BROWSER2_PROTOCOL    *This,
    IN  OUT UINTN                               *ResultsDataSize,
    IN  OUT EFI_STRING                          ResultsData,
    IN      BOOLEAN                            RetrieveData,
    IN      CONST EFI_GUID                     *VariableGuid, OPTIONAL
    IN      CONST CHAR16                       *VariableName OPTIONAL
);

```

Parameters**This**

A pointer to the *EFI_FORM_BROWSER2_PROTOCOL* instance.

ResultsDataSize

A pointer to the size of the buffer associated with *ResultsData*. On input, the size in bytes of *ResultsData*. On output, the size of data returned in *ResultsData*.

ResultsData

A string returned from an IFR browser or equivalent. The results string will have no routing information in them.

RetrieveData

A BOOLEAN field which allows an agent to retrieve (if *RetrieveData* = **TRUE**) data from the uncommitted browser state information or set (if *RetrieveData* = **FALSE**) data in the uncommitted browser state information.

VariableGuid

An optional field to indicate the target variable GUID name to use.

VariableName

An optional field to indicate the target human-readable variable name.

Description

This service is typically called by a driver's callback routine which was in turn called by the browser. The routine called this service in the browser to retrieve or set certain uncommitted state information that resides in the open formsets.

Status Codes Returned

EFI_SUCCESS	The results have been distributed or are awaiting distribution.
EFI_BUFFER_TOO_SMALL	The <i>ResultsDataSize</i> specified was too small to contain the results data.
EFI_UNSUPPORTED	Uncommitted browser state is not available at the current stage of execution.

35.7 HII Popup Protocol

35.7.1 EFI_HII_POPUP_PROTOCOL

Summary

This protocol provides services to display a popup window.

The protocol is typically produced by the forms browser and consumed by a driver's callback handler.

GUID

```
#define EFI_HII_POPUP_PROTOCOL_GUID \
{ 0x4311edc0, 0x6054, 0x46d4, { 0x9e, 0x40, 0x89, 0x3e, 0xa9, 0x52, 0xfc, 0xcc } }
```

Protocol Interface Structure

```
typedef struct {
    UINT64                Revision;
    EFI_HII_CREATE_POPUP  CreatePopup;
} EFI_HII_POPUP_PROTOCOL;
```

Parameters**Revision**

Protocol revision

CreatePopup

Displays a popup window

Related Definitions

```
#define *EFI_HII_POPUP_PROTOCOL_REVISION* 1
```

35.7.2 EFI_HII_POPUP_PROTOCOL.CreatePopup()

Summary

Displays a popup window.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_HII_CREATE_POPUP) (
    IN EFI_HII_POPUP_PROTOCOL      *This,
    IN EFI_HII_POPUP_STYLE         PopupStyle,
    IN EFI_HII_POPUP_TYPE          PopupType,
    IN EFI_HII_HANDLE              HiiHandle
    IN EFI_STRING_ID               Message,
    OUT EFI_HII_POPUP_SELECTION    *UserSelection OPTIONAL,
);
```

Parameters

This

A pointer to the *EFI_HII_POPUP_PROTOCOL* instance.

PopupStyle

Popup style to use. *EFI_HII_POPUP_STYLE* is defined in the “Related Definitions” below.

PopupType

Type of the popup to display. *EFI_HII_POPUP_TYPE* is defined in the “Related Definitions” below.

HiiHandle

HII handle of the string pack containing *Message*

Message

A message to display in the popup box.

UserSelection

User selection. *EFI_HII_POPUP_SELECTION* is defined in the “Related Definitions” below.

Description

The *CreatePopup()* function displays a modal message box that contains string specified by *Message*. Explicit line break characters can be used to specify a multi-line message (*Common Control Codes*). A popup window may contain user selectable options. The option selected by a user is returned via an optional *UserSelection* parameter.

A list of options presented to a user is defined by the *PopupType*.

The *PopupStyle* provides a hint to protocol implementation regarding nature of the message being displayed. The function may optionally use *PopupStyle* to customize visual appearance of the message box.

EfiHiiPopupTypeOk is a simple popup window with a single user selectable option that can be used to acknowledge the message. If *UserSelection* is specified, it is set to EfiHiiPopupSelectionOk.

EfiHiiPopupTypeOkCancel is a popup window with two user selectable options: OK and Cancel.

EfiHiiPopupTypeYesNo is a popup window with two user selectable options: Yes and No.

EfiHiiPopupTypeYesNoCancel is a popup window with three user selectable options: Yes, No, and Cancel.

Related Definitions

```
typedef enum {
    EfiHiiPopupStyleInfo,
    EfiHiiPopupStyleWarning,
    EfiHiiPopupStyleError
}    EFI_HII_POPUP_STYLE;
typedef enum {
    EfiHiiPopupTypeOk,
    EfiHiiPopupTypeOkCancel,
    EfiHiiPopupTypeYesNo,
    EfiHiiPopupTypeYesNoCancel
}    EFI_HII_POPUP_TYPE;
typedef enum {
    EfiHiiPopupSelectionOk,
    EfiHiiPopupSelectionCancel,
    EfiHiiPopupSelectionYes,
    EfiHiiPopupSelectionNo
}    EFI_HII_POPUP_SELECTION;
```

Status Codes Returned

<i>EFI_SUCCESS</i>	The popup box was successfully displayed
<i>EFI_INVALID_PARAMETER</i>	HiiHandle and Message do not define is a valid HII string.
<i>EFI_INVALID_PARAMETER</i>	PopupType is not one of the values defined by this specification.
<i>EFI_OUT_OF_RESOURCES</i>	There are not enough resources available to display the popup box.